

IUSIA

INTELLIGENCE. LAW. ADVANTAGE.

TECHNICAL ARCHITECTURE

Arquitectura Técnica del MVP

Cloudflare-first, multi-proveedor y preparada para evolución SaaS

PRODUCTO IUSIA	MARCA MATRIZ Go Legaltech	VERSIÓN v1.0
--------------------------	-------------------------------------	------------------------

MVP - Primera versión operativa y base de evolución multi-tenant

01 Principios de infraestructura

- Deploy continuo desde GitHub; infraestructura sin administración de servidor tradicional.
- Cloudflare aloja aplicación, API, estado transaccional, prompts de sistema y orquestación.
- Google Drive almacena documentos del usuario; IUSIA conserva metadatos, permisos, índices y referencias.
- OpenAI y Gemini se consumen mediante adapters del Core; ningún agente queda acoplado a un proveedor.
- Todo runtime de agentes debe ser auditable: ejecución, modelo, herramientas, costo, inputs y outputs.

DECISIÓN MVP
Cloudflare será la plataforma base. Evitamos cPanel y no introducimos Supabase en el MVP. Si el crecimiento multi-tenant exige PostgreSQL gestionado, se evalúa como evolución sin cambiar el contrato del Core.

02 Mapa lógico

Capa	Servicio	Responsabilidad
Web App	Cloudflare Workers/Pages compatible	Dashboard, casos, Intelligence, administración.
API / Core	Cloudflare Workers	Authz, cases, docs metadata, executions, providers, credits.
Workflow Engine	Cloudflare Workflows	DAG duradero, waits, retries, fan-out/fan-in, gates.
Async jobs	Cloudflare Queues	Procesamiento desacoplado: ingestión, callbacks, tareas extensas.
Concurrency state	Durable Objects (selectivo)	Locks de run, streaming/estado vivo cuando se requiera.
Database	D1	Usuarios, org, cases, ACL, tasks, metadata, executions, costs, ledgers.
System storage	R2	Prompts .md, specs, artifacts internos y archivos de sistema versionados.
User documents	Google Drive	Fuentes, anexos y entregables del usuario.
AI providers	OpenAI + Gemini adapters	Model routing por agente/tarea.
Secrets	Cloudflare Secrets	API keys, OAuth secrets y credenciales; nunca en repo/prompts.
Observability	Logs + custom audit tables	Errores, latencia, consumo, tracing y actividad.

03

Distribución de datos

Destino	Tipo	Contenido
D1	Estado transaccional	users, organizations, matters, ACL, tasks, issues, executions, usage, wallet ledger, audit events
R2	Sistema / IA	agent.md/instructions, workflow specs, schemas, outputs internos .md/json, snapshots versionados
Google Drive	Documentos del usuario	fuentes, anexos, entregables, carpetas de case
Proveedor IA	Procesamiento temporal	solo contexto mínimo requerido por cada ejecución; no es sistema de registro

REGLA DOCUMENTAL
La UI puede mostrar un único expediente lógico, aunque el archivo físico viva en Drive y la metadata/índice viva en D1. El usuario no debe gestionar dos sistemas.

04

Arquitectura de ejecución multiagente

Paso	Acción	Componente
1	API crea workflow_run y execution root	D1
2	Orchestrator prepara WorkPackage	Worker
3	Workflow lanza agentes elegibles	Workflow + provider adapters
4	Ramas independientes corren en paralelo	Workflow / Queues
5	Cada output se valida contra schema/gate	Worker validators
6	Fan-in espera dependencias requeridas	Workflow
7	OO integra y decide siguiente etapa	Agent Runtime
8	Resultado y costo quedan persistidos	D1 + R2/Drive según tipo

05

Model Router

CONTRATO NEUTRAL
Agent Runtime -> Model Router -> Provider Adapter. El agent definition define política (capacidad, costo, reasoning, fallback), no una llamada específica de OpenAI o Gemini.

Objeto	Campos esenciales
Agent policy	provider preference, model class, reasoning, tool permissions, timeout, retries
Provider call	provider, model, request id, input/output usage, latency, raw cost
Pricing layer	provider_cost -> platform markup -> IUSIA credits
Fallback	provider/model alternativo solo cuando la política lo permita

06 GitHub y despliegue

Rama	Entorno	Regla
main	Producción	Merge aprobado -> tests -> deploy automático a Cloudflare.
staging	Pruebas integradas	Branch/PR -> entorno preview/staging con secrets separados.
feature/*	Desarrollo	Claude/Codex trabajan por cambios acotados y PR revisable.

- No almacenar API keys en GitHub; usar Cloudflare Secrets y entornos separados.
- Migraciones D1 versionadas y repetibles.
- Deploy bloqueado si fallan tests críticos de permisos, tenancy, agent outputs o gates.
- Rollback por release anterior y migraciones compatibles hacia atrás cuando sea posible.

07 Seguridad mínima desde el MVP

Control	MVP
Aislamiento	organization_id y matter ACL en toda consulta; deny-by-default.
Google Drive	OAuth con scopes mínimos; carpeta vinculada por Drive Folder ID.
IA	Context minimization; cada agente recibe solo el WorkPackage necesario.
Tools	Permisos declarativos por agente; lectura/escritura diferenciadas.
Auditoría	Actor, acción, matter, resource, timestamp, before/after cuando aplique.
Secrets	Workers Secrets; nunca logs ni prompts.

08 Plan de activación hoy

1. Crear repositorio GitHub del producto y conectar proyecto Cloudflare.
2. Levantar shell web + autenticación + D1 schema mínimo.
3. Implementar users/roles, matters y ACL por case.
4. Conectar Google Drive en modo lectura/listado para un case piloto.
5. Implementar Agent Runtime + un adapter de proveedor primero; segundo adapter detrás del mismo contrato.
6. Ejecutar flujo piloto 00 -> 01 -> 03 -> especialista -> 06 y persistir execution ledger.

7. Añadir dashboard Director e Intelligence con datos reales del piloto.
8. Activar staging -> pruebas -> production deploy desde GitHub.

09 Decisiones diferidas

Tema	MVP	Reevaluación
Vector search	No requerido inicialmente	Cuando RAG sobre corpus crezca
PostgreSQL/Supabase	No	Cuando D1 o analytics multi-tenant lo justifiquen
Stripe / wallet recarga	Simular ledger interno	Después de validar uso y pricing
Multi-region advanced	Cloudflare default	Al escalar clientes/regiones
Full DMS	Google Drive	Solo si demanda del mercado lo exige